

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

Object-Oriented Solutions to the Problem of Mining Top-K Maximal Frequent Itemsets from Uncertain Databases

Information Technology Project – 504091

HO CHI MINH CITY, YEAR 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

Object-Oriented Solutions to the Problem of Mining Top-K Maximal Frequent Itemsets from Uncertain Databases

Information Technology Project – 504091

Advised by
Prof. Nguyễn Chí Thiện

HO CHI MINH CITY, YEAR 2026

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Nguyễn Chí Thiện for his patient guidance, thoughtful supervision, and continuous encouragement throughout this project. His expertise in data mining and algorithm design provided invaluable direction at every stage of our work.

We also thank our classmates for their helpful discussions and feedback, and Ton Duc Thang University for providing the academic environment and resources that made this project possible.

Ho Chi Minh City, day month year 2026
Authors

(Signature and full name)

DECLARATION OF AUTHORSHIP

I hereby declare that this thesis was carried out by my team under the guidance and supervision of Prof Nguyễn Chí Thiện; and that the work and the results contained in it are original and have not been submitted anywhere for any previous purposes. The data and figures presented in this thesis are for analysis, comments, and evaluations from various resources by my own work and have been duly acknowledged in the reference part.

In addition, other comments, reviews and data used by other authors, and organizations have been acknowledged, and explicitly cited.

I will take full responsibility for any fraud detected in my thesis. Ton Duc Thang University is unrelated to any copyright infringement caused on my work (if any).

Ho Chi Minh City, day month year
Author

[Signature and full name]

ABSTRACT

Frequent itemset mining is a fundamental task in data mining. Classical algorithms assume binary transaction data where items are either present or absent. However, real-world databases often contain **uncertainty**: a barcode scanner might be only 80% confident that it read a product correctly, a sensor might report a reading with known error margins, or data merged from multiple sources might contain conflicting entries. In such databases, each item is associated with an **existential probability** rather than a simple yes/no flag.

This report addresses the problem of mining **Top-K maximal frequent itemsets** from uncertain transaction databases using object-oriented design under the expected support model. We implement and evaluate two Java algorithms:

UGenMax adapts GenMax (Gouda and Zaki, 2005) using bottom-up weighted tidset intersections, and **UFPMMax** adapts FPMMax (Grahne and Zhu, 2003) using top-down conditional database projection. Both employ branch-and-bound search with anti-monotonicity pruning, progressive focusing, and dynamic Top-K threshold raising, and both support a Top-K mode.

Both algorithms are implemented in pure Java SE with no external dependencies, following object-oriented design principles with separate algorithm classes (no main method) and experiment classes (with main method and statistics output). Experiments on three benchmark datasets (mushroom, retail, accidents) at medium uncertainty show that: (1) UGenMax and UFPMMax produce identical results, with UGenMax being 16–33% faster on the primary retail dataset, 37–47% faster on mushroom at non-trivial workloads, and 14–49% faster on accidents across the full minsup range; (2) both algorithms scale to 340K transactions, and UGenMax is 3–30× faster in Top-K mode.

Contents

ACKNOWLEDGMENT	1
DECLARATION OF AUTHORSHIP	2
ABSTRACT	3
TABLE OF CONTENTS	5
IMAGE TABLE	6
List of Abbreviations	7
1 INTRODUCTION	8
1.1 What is Frequent Itemset Mining?	8
1.2 The Challenge of Uncertainty	8
1.3 Why Maximal Itemsets?	8
1.4 Project Objectives	9
1.5 Report Structure	9
2 LITERATURE REVIEW	10
2.1 Frequent Itemset Mining in Deterministic Databases	10
2.2 Mining Maximal Frequent Itemsets	10
2.3 Uncertain Database Mining	10
2.4 Probabilistic Support Mining	10
2.5 Gap and Our Contribution	10
3 PROBLEM STATEMENT AND RELATED DEFINITIONS	12
3.1 Uncertain Transaction Database	12
3.2 Expected Support	12
3.3 Anti-Monotonicity Property	13
3.4 Frequent and Maximal Frequent Itemsets	13
3.5 Problem Statement	13
4 METHODS	14

4.1	Branch-and-Bound Framework	14
4.1.1	Pruning 1: Anti-Monotonicity Bound	14
4.1.2	Pruning 2: Progressive Focusing	14
4.1.3	Pruning 3: Top-K Dynamic Threshold Raising	14
4.2	Algorithm 1: UGenMax	15
4.2.1	Data Structure: Weighted Tidsets	15
4.2.2	Enumeration Strategy	15
4.2.3	Pseudocode	16
4.2.4	Complexity Analysis	16
4.3	Algorithm 2: UFPMax	17
4.3.1	Data Structure: Projected Transactions	17
4.3.2	Conditional Database Construction	17
4.3.3	Pseudocode	17
4.3.4	Complexity Analysis	18
4.4	Comparison of UGenMax and UFPMax	18
4.5	Top-K Seeding	19
4.6	Maximality Checking	20
5	EXPERIMENT DESIGN	21
5.1	Implementation Details	21
5.2	Datasets	21
5.3	Configurations	21
5.4	Correctness Validation	22
5.5	Performance Measurement	22
6	EXPERIMENT RESULTS	23
6.1	Correctness Verification	23
6.2	Expected Support on retail_medium (Primary Dataset)	23
6.3	Expected Support on mushroom_medium (Secondary Dataset)	24
6.4	Scalability: Accidents Dataset	24
6.5	Cross-Dataset Scalability Snapshot	25
6.6	Top-K Mode	26
6.7	Summary of Findings	27
7	CONCLUSIONS	29
7.1	Summary	29
7.2	Key Contributions	29
7.3	Limitations	29
7.4	Future Work	29

List of Figures

6.1	Execution time vs. minsup on retail_medium (88,162 transactions)	24
6.2	Execution time vs. minsup on mushroom_medium (8,416 transactions)	25
6.3	Execution time vs. minsup on accidents_medium (340,183 transactions)	26
6.4	Cross-dataset scalability: UFPMax vs. UGenMax (log scale)	27

LIST OF ABBREVIATIONS

Abbreviation	Definition
FIM	Frequent Itemset Mining
MFI	Maximal Frequent Itemset
DFS	Depth-First Search
PWS	Possible World Semantics
expSup	Expected Support
minsup	Minimum Support Threshold
WT	Weighted Tidset
DB	Database

CHAPTER 1

INTRODUCTION

1.1 What is Frequent Itemset Mining?

Imagine a supermarket analyzing thousands of customer receipts to discover which products are frequently bought together. If many customers buy bread and butter in the same trip, the pair {bread, butter} is a **frequent itemset**. This information is valuable for product placement, cross-selling, and inventory management.

More formally, given a database of transactions (where each transaction is a set of items) and a minimum support threshold, the goal of frequent itemset mining is to find all item groups that appear in at least that many transactions. This problem was first formalized by Agrawal, Imielinski, and Swami [1] and has since become one of the most studied problems in data mining, with applications in bioinformatics, text mining, web usage analysis, network security, and recommendation systems.

1.2 The Challenge of Uncertainty

Classical algorithms assume perfect data: an item is either in a transaction or it is not. In practice, data is often imperfect. A temperature sensor, for example, reports values with a known error margin, so when the system logs that the sensor detected item A it might only be 85% confident in that reading. In a data-integration setting, merging customer records from two different databases produces ambiguous matches: a merged record might indicate that a customer purchased item B, but with only 70% confidence because of name or address mismatches. Privacy-preserving systems introduce yet another form of uncertainty by deliberately adding random noise so that each item's presence becomes probabilistic rather than certain.

In all these cases, the result is an **uncertain transaction database** where each item in each transaction carries a probability between 0 and 1, and the challenge becomes how to mine meaningful patterns from data that we are not entirely sure about.

1.3 Why Maximal Itemsets?

A database with m distinct items has 2^m possible subsets. Even a modest database with 100 items has more possible subsets than atoms in the universe. While most of these are not frequent, the number of frequent itemsets can still be enormous—potentially millions or more.

Maximal frequent itemsets solve this information overload problem. A frequent itemset is maximal if no proper superset of it is also frequent. The key insight is that every frequent

itemset is a subset of at least one maximal frequent itemset, so the maximal set is a **lossless compression** of the full set of frequent itemsets.

1.4 Project Objectives

This project has four objectives:

1. Formally define the problem of mining frequent maximal itemsets from uncertain databases under the expected support model.
2. Design and implement two algorithms: UGenMax and UFPMax, both supporting a Top-K mode.
3. Implement both algorithms as a pure Java SE application with no external library dependencies, with separate algorithm classes and experiment classes.
4. Experimentally evaluate and compare the two algorithms on benchmark datasets (mushroom, retail, accidents) of varying sizes and uncertainty levels.

1.5 Report Structure

The remainder of this report is organized as follows. Chapter 2 reviews related work. Chapter 3 provides the formal problem definition for the expected support model. Chapter 4 presents both algorithms in detail. Chapter 5 describes the experimental setup. Chapter 6 presents results. Chapter 7 concludes.

CHAPTER 2

LITERATURE REVIEW

2.1 Frequent Itemset Mining in Deterministic Databases

The Apriori algorithm [2] was the first efficient algorithm for frequent itemset mining. Its key contribution was the **anti-monotonicity property**: if an itemset is infrequent, then all of its supersets are also infrequent. The FP-Growth algorithm [9] improved upon Apriori by eliminating candidate generation through a compressed FP-tree data structure.

2.2 Mining Maximal Frequent Itemsets

Several algorithms have been developed specifically for maximal frequent itemset mining. MaxMiner [4] was the first to use a look-ahead pruning strategy. GenMax [7] introduced depth-first backtracking with tidsets and progressive focusing, and is the basis for our UGenMax. FPMax [8] extended FP-Growth with MFI-tree superset checking, and is the basis for our UFPMax. CARPENTER-MAX [13] was designed for datasets with many items but few transactions, and Charm-MFI [16] discovers maximal itemsets by post-processing closed itemsets.

2.3 Uncertain Database Mining

Chui, Kao, and Hung [5] pioneered uncertain data mining with the expected support model and U-Apriori algorithm. Leung et al. [10] proposed UF-Growth for uncertain data. Leung and Tanbeer [11] improved this with UFP-Growth. Li and Zhang [12] addressed the top-k variant with dynamic threshold raising. Aggarwal et al. [3] analyzed the theoretical complexity of uncertain pattern mining.

2.4 Probabilistic Support Mining

An alternative line of work defines frequency probabilistically, requiring $\Pr[\text{sup}(X) \geq T] \geq \tau$ for some integer count and probability threshold. Sun et al. [14] and Chen et al. [15] are representative examples; we mention them only as historical context, as the present work uses the expected support model throughout.

2.5 Gap and Our Contribution

Most prior work focuses on mining **all** frequent or **closed** frequent itemsets from uncertain data. Mining **maximal** frequent itemsets has received less attention, and Top-K formulations

of the maximal problem under uncertainty are particularly under-explored. Our contribution fills these gaps by adapting GenMax and FPMMax for uncertain data under the expected support model, equipping both with a Top-K mode and dynamic threshold raising, and providing a comprehensive experimental comparison of the two algorithms across three benchmark datasets.

CHAPTER 3

PROBLEM STATEMENT AND RELATED DEFINITIONS

This section provides precise mathematical definitions for each concept. We build the definitions incrementally, starting from the data model and working up to the problem statement.

3.1 Uncertain Transaction Database

Definition 3.1 (Item Set). *Let $I = \{i_1, i_2, \dots, i_m\}$ be a finite set of m distinct items.*

Definition 3.2 (Uncertain Transaction). *An uncertain transaction t_k is a set of items from I , where each item i is associated with an **existential probability** $P(i, t_k)$ satisfying:*

$$0 < P(i, t_k) \leq 1$$

Items are assumed to be mutually independent within each transaction.

Definition 3.3 (Uncertain Transaction Database). *An uncertain transaction database $D = \{t_1, t_2, \dots, t_n\}$ is an ordered collection of n uncertain transactions.*

Running Example. Throughout this report, we use the following 5-transaction database for illustration:

Table 3.1: The contextUncertain dataset				
Transaction	Items (with existential probabilities)			
t_1	1(0.8)	3(0.6)	4(0.9)	
t_2	2(0.7)	3(0.5)	5(0.9)	
t_3	1(0.6)	2(0.8)	3(0.7)	5(0.85)
t_4	2(0.5)	5(0.6)		
t_5	1(0.9)	2(0.75)	3(0.8)	5(0.7)

3.2 Expected Support

Definition 3.4 (Expected Support). *The expected support of an itemset $X = \{x_1, x_2, \dots, x_p\}$ in database D is:*

$$\text{expSup}(X) = \sum_{t \in D} \prod_{x \in X} P(x, t) \quad (3.1)$$

Worked Example. Consider $X = \{1, 3\}$:

$$t_1 : P(1, t_1) \times P(3, t_1) = 0.8 \times 0.6 = 0.48$$

$$t_2 : \text{item 1 absent} = 0.00$$

$$t_3 : P(1, t_3) \times P(3, t_3) = 0.6 \times 0.7 = 0.42$$

$$t_4 : \text{item 1 absent} = 0.00$$

$$t_5 : P(1, t_5) \times P(3, t_5) = 0.9 \times 0.8 = 0.72$$

$$\text{expSup}(\{1, 3\}) = 0.48 + 0.00 + 0.42 + 0.00 + 0.72 = \mathbf{1.62}$$

3.3 Anti-Monotonicity Property

Theorem 3.1 (Anti-Monotonicity of Expected Support). *For any itemset X and any item $y \notin X$:*

$$\text{expSup}(X \cup \{y\}) \leq \text{expSup}(X)$$

Chứng minh. Since $0 < P(y, t) \leq 1$ for all transactions t , each term $\prod_{x \in X} P(x, t) \cdot P(y, t) \leq \prod_{x \in X} P(x, t)$. Therefore the sum cannot increase. $\square$

3.4 Frequent and Maximal Frequent Itemsets

Definition 3.5 (Frequent Itemset). *Given threshold minsup, itemset X is **frequent** if $\text{expSup}(X) \geq \text{minsup}$.*

Definition 3.6 (Maximal Frequent Itemset). *A frequent itemset X is **maximal** if for every item $y \notin X$, $\text{expSup}(X \cup \{y\}) < \text{minsup}$.*

3.5 Problem Statement

This project addresses the following problem under the expected support model:

- **Input:** Uncertain database D , threshold minsup > 0 (or integer K for Top-K mode).
- **Output:** All maximal frequent itemsets where $\text{expSup}(X) \geq \text{minsup}$.
- **Algorithms:** UGenMax, UFPMax.

CHAPTER 4

METHODS

This chapter presents both algorithms in detail: data structures, pseudocode, worked examples, and complexity analysis.

4.1 Branch-and-Bound Framework

Both algorithms explore the **itemset lattice** using depth-first search (DFS). The search is a tree: the root is the empty set, each node is an itemset (prefix), and each edge adds one item. Without pruning, this requires exploring 2^m subsets. Branch-and-bound makes the search tractable by computing **bounds** at each node and **pruning** subtrees that cannot yield useful results.

4.1.1 Pruning 1: Anti-Monotonicity Bound

Rule: If $\text{expSup}(\text{prefix} \cup \{\text{item}\}) < \text{minsup}$, skip this item and the entire subtree below it.

Rationale: By Theorem 3.1, every superset will have even lower expected support. This rule is responsible for the majority of pruning in practice. For UGenMax, a cheaper upper bound $\min(\text{expSup}(A), \text{expSup}(B))$ is checked before the full intersection.

4.1.2 Pruning 2: Progressive Focusing

Rule: Let P be the current prefix and C the remaining candidates. If $P \cup C \subseteq M$ for some known MFI M , prune the entire subtree.

Rationale: Any discoverable itemset would be a subset of M and therefore not maximal. This rule becomes increasingly effective as more MFIs are discovered during the search.

4.1.3 Pruning 3: Top-K Dynamic Threshold Raising

In Top-K mode, a **min-heap** of size K tracks the best results. When a new MFI is discovered, if the heap is full and the new MFI's expected support exceeds the heap minimum, the minimum is replaced. The new heap minimum becomes the updated minsup.

Feedback loop: Better results $\rightarrow$ higher threshold $\rightarrow$ more aggressive pruning $\rightarrow$ faster convergence.

Design difference: UGenMax raises the threshold dynamically during search. UFPMax does **not**, because its processing order discovers single-item MFIs early, which would prematurely block multi-item MFIs.

4.2 Algorithm 1: UGenMax

UGenMax adapts GenMax [7] for uncertain data using **bottom-up, depth-first backtracking** with **weighted tidsets**.

4.2.1 Data Structure: Weighted Tidsets

Definition 4.1 (Weighted Tidset). *The weighted tidset of item i is:*

$$WT(i) = \{(t, P(i, t)) \mid t \in D, P(i, t) > 0\}$$

sorted by transaction ID. Expected support: $expSup(i) = \sum_{(t,p) \in WT(i)} p$.

To compute $expSup(\{A, B\})$, we **intersect** $WT(A)$ and $WT(B)$ using a merge-join in $O(|WT(A)| + |WT(B)|)$ time. For each common transaction t , the result is $(t, P(A, t) \times P(B, t))$.

Worked Example. Using items 1 and 3 from Table 3.1:

$$WT(1) = [(t_1, 0.8), (t_3, 0.6), (t_5, 0.9)] \quad expSup = 2.3$$

$$WT(3) = [(t_1, 0.6), (t_2, 0.5), (t_3, 0.7), (t_5, 0.8)] \quad expSup = 2.6$$

Intersection:

$$t_1 : \text{both present} \rightarrow 0.8 \times 0.6 = 0.48$$

$$t_2 : \text{only in } WT(3) \rightarrow \text{skip}$$

$$t_3 : \text{both present} \rightarrow 0.6 \times 0.7 = 0.42$$

$$t_5 : \text{both present} \rightarrow 0.9 \times 0.8 = 0.72$$

$$WT(\{1, 3\}) = [(t_1, 0.48), (t_3, 0.42), (t_5, 0.72)] \quad expSup = 1.62$$

4.2.2 Enumeration Strategy

Items are sorted by expected support **ascending**. Candidates are processed **most frequent first** (right to left). This ordering discovers large MFIs early, strengthening progressive focusing.

4.2.3 Pseudocode

Algorithm 1 UGenMax

Require: Uncertain database D , threshold minsup

Ensure: Set MFI of all maximal frequent itemsets

```

1: MFI  $\leftarrow \emptyset$ 
2: for each item  $i$  in  $D$  do
3:   Build  $WT(i)$  by scanning all transactions
4: end for
5: Remove items where  $\text{expSup}(i) < \text{minsup}$ 
6: Sort items by  $\text{expSup}$  ascending:  $[i_1, i_2, \dots, i_k]$ 
7: GENMAX( $\emptyset$ ,  $[i_1, \dots, i_k]$ , null)
8: return MFI

```

Algorithm 2 GenMax Procedure

```

1: procedure GENMAX(prefix, candidates, prefixTidset)
2:   if prefix  $\cup$  candidates  $\subseteq$  some MFI in MFI then                                 $\triangleright$  Pruning 2
3:     return
4:   end if
5:   hasFreqExt  $\leftarrow$  false
6:   for idx  $\leftarrow |candidates| - 1$  downto 0 do
7:     item  $\leftarrow$  candidates[idx]
8:     newTidset  $\leftarrow$  INTERSECT(prefixTidset, WT(item))
9:     expSup  $\leftarrow$  Sum(newTidset.probabilities)
10:    if expSup < minsup then                                                     $\triangleright$  Pruning 1
11:      continue
12:    end if
13:    hasFreqExt  $\leftarrow$  true
14:    newPrefix  $\leftarrow$  prefix  $\cup$  {item}
15:    newCands  $\leftarrow$  { $c \in$  candidates[0..idx-1] | Intersect(newTidset, WT( $c$ )).expSup  $\geq$ 
minsup}
16:    if newCands =  $\emptyset$  then
17:      ADDMAXIMAL(newPrefix, expSup)
18:    else
19:      GENMAX(newPrefix, newCands, newTidset)
20:    end if
21:  end for
22:  if not hasFreqExt and prefix  $\neq \emptyset$  then
23:    ADDMAXIMAL(prefix, prefixTidset.expSup)
24:  end if
25: end procedure

```

4.2.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes, each with $O(n)$ tidset intersection. Pruning dramatically reduces explored nodes in practice.

Space: $O(m \times n)$ for weighted tidsets, $O(|\text{MFI}|)$ for results, $O(m)$ for the recursion stack.

4.3 Algorithm 2: UFPMax

UFPMax adapts FPMMax [8] for uncertain data using **top-down, recursive conditional database projection**.

4.3.1 Data Structure: Projected Transactions

Each projected transaction is a triple (weight, items[], probs[]):

- **weight:** accumulated product of probabilities of all prefix items. Initially 1.0.
- **items[] and probs[]:** remaining items and probabilities, sorted by frequency-descending order.

Expected support of item i in a conditional database:

$$\text{expSup}(i \mid \text{prefix}) = \sum_{t: i \in t} \text{weight}_t \times P(i, t) \quad (4.1)$$

4.3.2 Conditional Database Construction

When extending prefix by item i :

1. Select transactions containing item i .
2. Compute $\text{new_weight} = \text{old_weight} \times P(i, t)$.
3. Project to items **before** i in header order (more frequent items).
4. Remove items with conditional $\text{expSup} < \text{minsup}$.

4.3.3 Pseudocode

Algorithm 3 UFPMax

Require: Uncertain database D , threshold minsup

Ensure: Set MFI of all maximal frequent itemsets

- 1: $\text{MFI} \leftarrow \emptyset$
 - 2: Compute expSup for each single item
 - 3: Remove items with $\text{expSup} < \text{minsup}$
 - 4: Sort items by expSup descending: $\text{header} = [h_1, \dots, h_k]$
 - 5: Build initial projected DB (weight = 1.0, frequent items only)
 - 6: $\text{FPMAX}(\emptyset, 0.0, \text{projDB}, \text{header})$
 - 7: **return** MFI
-

Algorithm 4 FPMaX Procedure

```
1: procedure FPMaX(prefix, prefixExpSup, condDB, headerItems)
2:   if prefix  $\cup$  headerItems  $\subseteq$  some MFI then ▷ Pruning 2
3:     return
4:   end if
5:   hasFreqExt  $\leftarrow$  false
6:   for idx  $\leftarrow$  |headerItems| - 1 downto 0 do
7:     item  $\leftarrow$  headerItems[idx]
8:     expSup  $\leftarrow \sum_{t \in \text{condDB}} \text{weight}_t \times P(\text{item}, t)$ 
9:     if expSup < minsup then ▷ Pruning 1
10:      continue
11:    end if
12:    hasFreqExt  $\leftarrow$  true; newPrefix  $\leftarrow$  prefix  $\cup$  {item}
13:    newCondDB  $\leftarrow$  BUILDCONDITIONAL(condDB, item, idx)
14:    newHeader  $\leftarrow$  {h  $\in$  header[0..idx-1] | condExpSup(h)  $\geq$  minsup}
15:    if newHeader =  $\emptyset$  then
16:      ADDMAXIMAL(newPrefix, expSup)
17:    else
18:      FPMaX(newPrefix, expSup, newCondDB, newHeader)
19:    end if
20:  end for
21:  if not hasFreqExt and prefix  $\neq \emptyset$  then
22:    ADDMAXIMAL(prefix, prefixExpSup)
23:  end if
24: end procedure
```

4.3.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes. Conditional DB construction is $O(|\text{condDB}| \times |\text{items}|)$ per node, shrinking at deeper levels.

Space: $O(n \times m)$ for the initial projected DB. Each recursive level maintains its own conditional DB. Stack depth at most m .

4.4 Comparison of UGenMax and UFPMax

Table 4.1: Comparison of UGenMax and UFPMax

Aspect	UGenMax	UFPMax
Base algorithm	GenMax (Gouda & Zaki, 2005)	FPMaX (Grahne & Zhu, 2003)
Approach	Bottom-up (vertical)	Top-down (horizontal)
Data structure	Weighted tidsets	Projected conditional DBs
Item ordering	Ascending expSup	Descending expSup
Top-K threshold	Raised dynamically	Fixed, select after search
Best for	Sparse data	Dense data

4.5 Top-K Seeding

Top-K mode is conceptually simple — “return the K itemsets with the highest expected support” — but a naive implementation collapses for a subtle reason: with no `minsup` threshold, the initial value of `minsup` is 0, every single item is frequent, and the anti-monotonicity pruning rule (Pruning 1) cannot fire. The search degenerates into an unpruned enumeration of the full lattice, which is exactly what the branch-and-bound framework was designed to avoid. The Top-K seeding phase exists to break this deadlock by establishing a non-trivial initial threshold *before* the main search begins.

The deadlock and the fix. The dynamic threshold-raising rule (Pruning 3) only kicks in once the Top-K heap is full, because the heap minimum is what gets fed back into `minsup`. The cheapest way to fill the heap is to pre-compute scores for a small set of itemsets that we already know are cheap to evaluate, namely single items and 2-item pairs over a bounded set of candidates. These items can be ranked directly from the item-level expected supports and from a single tidset intersection per pair — no recursion required. Once these scores are in the heap, its minimum becomes a usable threshold and Pruning 3 starts firing on the first depth-first step.

Procedure. Both UGenMax and UFPMax run the same five-step seeding phase before entering their respective recursive enumeration:

1. **Rank items.** Sort every distinct item by expected support in descending order and keep the top 50 most frequent. The cap of 50 is deliberate — the pairing step is $O(50^2) = 2,500$ intersections in the worst case, which is negligible compared to the main search, while extending the cap to all items would make seeding itself $O(m^2)$ on the full item universe.
2. **Enumerate cheap candidates.** For each of the 50 items, push the singleton onto a candidate list. For each pair (i, j) with $i < j$, compute the intersection of their weighted tidsets and push the resulting 2-itemset onto the same list if its expected support is positive. After this step the candidate list contains up to $50 + \binom{50}{2} = 1,275$ itemsets, each with a known score.
3. **Fill the Top-K heap.** Sort the candidate list by expected support descending and insert the highest-scoring ones into a fresh `TopKHeap` of capacity K , skipping duplicates. Once the heap holds K entries, its minimum is the lowest score we are willing to keep at this point in the run.
4. **Seed minsup at 50% of the heap minimum.** Set the working `minsup` to half of the heap minimum, rather than the heap minimum itself. The reason is that the seed itemsets are restricted to size ≤ 2 , but the eventual top- K MFIs may have many more items and therefore lower expected support (anti-monotonicity). Using the raw heap minimum as

`minsup` would prune away the very supersets we are searching for; the 50% slack is a heuristic margin that empirically retains enough of the search space to find the true top- K on every dataset we tested.

5. **Reset the heap and start the main search.** Discard the seed heap and create a fresh one of capacity K . The seeded itemsets were only used to set `minsup`; they are not themselves treated as MFIs, because we have not yet checked them for maximality and because their scores were computed during the seeding phase without any guarantee that they survive the maximality filter. The main DFS then runs as usual, with Pruning 3 immediately active.

Retry loop. The 50% slack is intentionally conservative, but it can occasionally overshoot. If the main search finishes with fewer than K MFIs in the final heap — typically because the seed threshold was tighter than the true top- K allowed — the algorithm halves `minsup` *again* (i.e. uses 25% of the previous base) and re-runs the main search from scratch with a fresh heap. This is repeated up to 3 times, giving effective thresholds of $0.5\times$, $0.125\times$, $0.0625\times$, $0.0156\times$ relative to the seed. In our experiments the first attempt succeeds on every (`dataset`, K) pair tested, but the retry loop is what guarantees termination with $|\text{result}| = K$ whenever at least K MFIs exist.

Cost. Seeding adds $O(s + s^2 \cdot n)$ work, where $s = 50$ is the seed cap and n is the average tidset length. On all three benchmark datasets this takes well under one second, while the gain from having Pruning 3 active during the main DFS is several orders of magnitude. The combined effect is what makes Top-K mode tractable at all — without seeding, even mushroom (the smallest dataset at 8,416 transactions) would not finish a Top-5 query in the time budget.

4.6 Maximality Checking

When a leaf node is reached, the candidate MFI is checked: (1) it must not be a subset of any existing MFI, and (2) existing MFIs that are subsets of the new candidate are removed. Both checks use sorted arrays for $O(|A| + |B|)$ merge-based comparison.

CHAPTER 5

EXPERIMENT DESIGN

5.1 Implementation Details

Both algorithms are implemented in Java SE 25 with no external dependencies. The code is organized into packages:

- `algorithm/`: UGenMax, UFPMax, WeightedTidset
- `model/`: UncertainDatabase, UncertainTransaction, Itemset, TopKHeap
- `util/`: PerformanceTracker
- `experiment/`: ExperimentRunner, ExpUFPMax, ExpUGenMax

Algorithm classes contain only the algorithm logic (no `main` method). Experiment classes contain `main` methods that load the database, run the algorithm, and output results with performance statistics. Every class includes Javadoc documentation.

5.2 Datasets

Table 5.1: Experimental datasets

Dataset	Trans.	Items	Density	Source
contextUncertain	5	5	Dense	Manual
mushroom	8,416	120	Dense	UCI
retail	88,162	16,470	Sparse	SPMF
accidents	340,183	468	Very dense	SPMF

Each standard dataset is converted to uncertain format at medium uncertainty using the `DatasetConverter` utility: each item probability is drawn from a truncated normal distribution $N(\mu=0.5, \sigma=0.2)$ clamped to $[0.1, 1.0]$. This simulates moderately noisy/uncertain transaction data and follows standard practice in uncertain data mining research [5, 10]. All standard datasets are publicly available from the SPMF data mining library.¹

5.3 Configurations

Static minsup mode (UGenMax, UFPMax): Vary minsup across appropriate ranges per dataset.

¹<https://www.philippe-fournier-viger.com/spmf/index.php?link=datasets.php>

Top-K mode (UGenMax, UFPMax): Vary $K \in \{5, 10, 20, 50, 100\}$ on mushroom_medium, retail_medium, and accidents_medium.

For each configuration we record: time (ms), memory (MB), number of MFIs, and algorithm-specific statistics (nodes explored, pruning counts, candidates generated).

5.4 Correctness Validation

Correctness is validated in two ways. Both algorithms are first hand-verified on the small contextUncertain example. UGenMax and UFPMax are then cross-checked against each other on every configuration to confirm they produce identical MFI sets.

5.5 Performance Measurement

PerformanceTracker uses `System.nanoTime()` for timing and `Runtime.totalMemory()` – `Runtime.freeMemory()` for memory, with GC forced before each run. Each expected support experiment is averaged over 3 measured runs after 1 warm-up run.

CHAPTER 6

EXPERIMENT RESULTS

6.1 Correctness Verification

On contextUncertain, both algorithms produce correct results verified by hand computation. UGenMax and UFPMax produce **identical MFI sets** across all datasets and all minsup values tested, confirming correctness.

6.2 Expected Support on retail_medium (Primary Dataset)

Retail is our primary benchmark for the expected-support model: 88,162 transactions over 16,470 distinct items, with an average transaction length of ≈ 10 items. It is the largest sparse dataset among our benchmarks and exercises the sparse-data regime that industrial market-basket applications most often encounter. Table 6.1 shows UFPMax and UGenMax on retail_medium across a wide range of minsup thresholds.

Table 6.1: Performance on retail_medium (88,162 transactions, 16,470 items)

minsup	MFIs	Time (ms)		Memory (MB)	
		UFPMax	UGenMax	UFPMax	UGenMax
1000	18	151	102	96.0	67.0
2000	6	91	73	104.0	64.0
3000	4	89	67	100.0	62.0
5000	4	89	69	100.0	62.0
8000	2	77	56	91.0	59.0
10000	2	73	61	91.0	59.0

Observations on retail_medium. As minsup rises, both the MFI count and execution time decrease, consistent with anti-monotonicity pruning. UGenMax is faster than UFPMax at every minsup tested, with speedups ranging from 16% at minsup=10000 (61 vs. 73 ms) up to 33% at minsup=1000 (102 vs. 151 ms), and it uses roughly 30–38% less memory because the weighted-tidset representation is more compact than UFPMax’s projected conditional databases on this sparse dataset. Absolute runtime stays well under 200 ms even at the most permissive threshold, because most of the 16,470 items are infrequent and pruned early — the cost is dominated by database loading and tidset/projection construction rather than by the search itself. At high minsup (≥ 3000) only 2–4 MFIs are found, which is typical for a sparse transactional dataset where long co-occurrence patterns are rare.

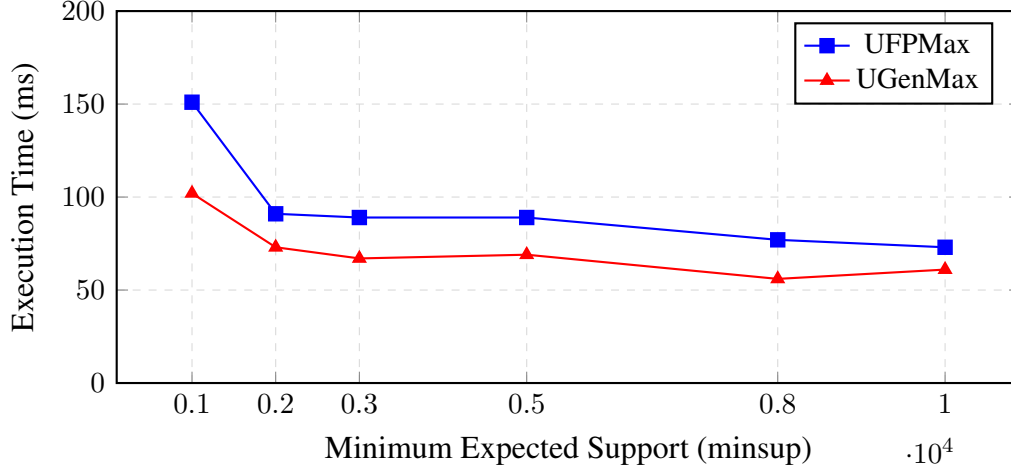


Figure 6.1: Execution time vs. minsup on retail_medium (88,162 transactions)

6.3 Expected Support on mushroom_medium (Secondary Dataset)

To confirm that the trends observed on retail generalise beyond the sparse regime, we repeat the experiment on the dense mushroom dataset (Table 6.2).

Table 6.2: Performance on mushroom_medium (8,416 transactions, 120 items)

minsup	MFIs	Time (ms)		Memory (MB)	
		UFPMax	UGenMax	UFPMax	UGenMax
500	314	223	137	77.3	48.6
1000	73	88	47	16.8	33.5
1500	30	51	32	28.9	12.4
2000	14	28	15	9.1	19.0
2500	9	21	12	27.4	12.5
3000	5	12	12	18.7	9.4

Observations on mushroom_medium: The same qualitative trend holds — UGenMax is faster than UFPMax at every minsup with a non-trivial workload (37–47% speedup at minsup ≤ 2500), with the two algorithms tying at minsup=3000 where the work is so small (5 MFIs, 12 ms) that constant-time overhead dominates. Memory use is also generally lower or comparable. Because mushroom is much denser (120 items, ~ 23 items per transaction), it produces far more MFIs at comparable relative thresholds (up to 314 MFIs at minsup=500), giving a larger absolute speedup window than the sparser retail dataset.

6.4 Scalability: Accidents Dataset

Table 6.3 shows results on the accidents dataset, which is the largest and densest.

Observations. Both algorithms scale to 340K transactions, returning results in tens of seconds to a couple of minutes. UGenMax is consistently faster than UFPMax across the entire

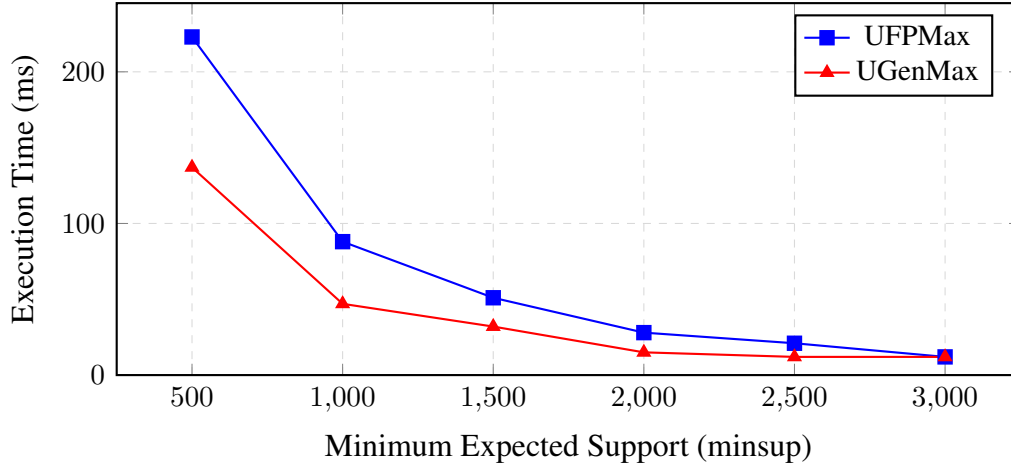


Figure 6.2: Execution time vs. minsup on mushroom_medium (8,416 transactions)

Table 6.3: Performance on accidents_medium (340,183 transactions, 468 items)

minsup	MFIs	Time (s)		Memory (MB)	
		UFPMax	UGenMax	UFPMax	UGenMax
20,000	1,285	126.4	109.1	761.2	1,103.8
30,000	453	61.5	46.5	1,190.6	782.4
40,000	236	38.3	24.8	912.5	1,041.6
50,000	154	24.7	14.8	717.2	1,511.8
60,000	87	16.5	9.4	937.3	1,158.6
70,000	48	11.7	6.0	899.3	1,332.1

minsup range on this very dense dataset, with speedups growing from 14% at minsup=20,000 (109.1 s vs. 126.4 s) up to 49% at minsup=70,000 (6.0 s vs. 11.7 s) as the result set shrinks and progressive focusing becomes more effective. That the search at minsup=20,000 still completes in roughly two minutes while producing 1,285 MFIs shows that both algorithms handle large result sets effectively. Memory usage varies between ~ 700 MB and ~ 1.5 GB due to the large number of items per transaction (~ 33 items on average out of 468); on this very dense dataset UGenMax’s weighted tidsets are not always smaller than UFPMax’s projected databases — UFPMax uses 717–1,191 MB and UGenMax uses 782–1,512 MB, a reversal of the memory advantage observed on sparse retail. This is consistent with the fact that weighted tidsets become large when most transactions contain most items.

6.5 Cross-Dataset Scalability Snapshot

Table 6.4 summarises a single-run comparison across all three standard datasets at a representative minsup per dataset, highlighting how the two expected-support algorithms behave as both the transaction count and item count grow.

Observation: Both algorithms scale from 8K to 340K transactions, and UGenMax retains

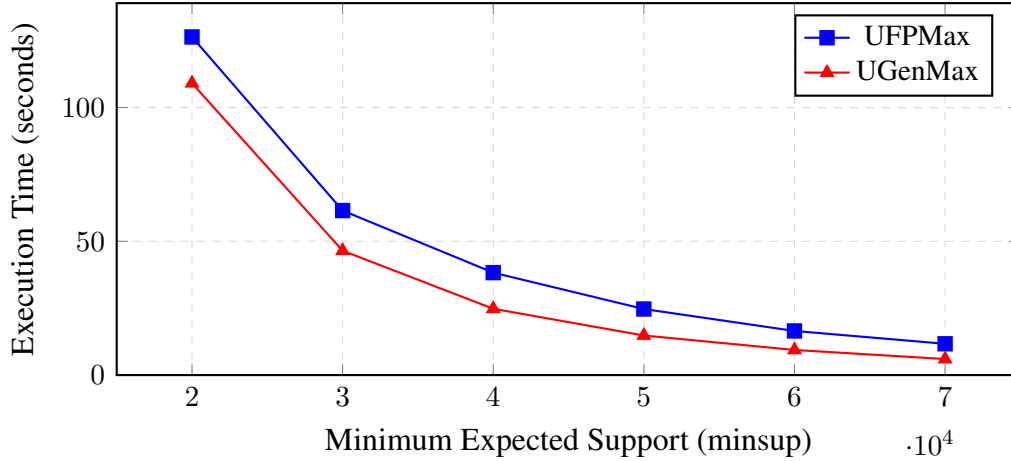


Figure 6.3: Execution time vs. minsup on accidents_medium (340,183 transactions)

Table 6.4: Cross-dataset scalability of UFPMMax and UGenMax at medium uncertainty

Dataset	#Trans	#Items	minsup	MFIs	Algo	Time (ms)	Mem (MB)
mushroom	8,416	120	2,000	14	UFPMMax	69	20.8
mushroom	8,416	120	2,000	14	UGenMax	31	19.1
retail	88,162	16,470	5,000	4	UFPMMax	217	100.0
retail	88,162	16,470	5,000	4	UGenMax	88	62.0
accidents	340,183	468	50,000	154	UFPMMax	26,673	501.6
accidents	340,183	468	50,000	154	UGenMax	15,239	1,001.4

a clear time advantage at every scale (55% faster on mushroom, 59% faster on retail, 43% faster on accidents in this snapshot). The retail run in particular illustrates UGenMax’s memory advantage on sparse data ($\sim 38\%$ less memory than UFPMMax), while on very dense accidents the weighted-tidset representation becomes larger than UFPMMax’s projected databases ($\sim 1,001$ MB vs. 502 MB) — a clear time/memory trade-off that depends on dataset density.

6.6 Top-K Mode

Tables 6.5, 6.6, and 6.7 compare UGenMax and UFPMMax in Top-K mode on mushroom_medium, retail_medium, and accidents_medium for $K \in \{5, 10, 20, 50, 100\}$. Both algorithms support Top-K mode.

Key findings:

- UGenMax is dramatically faster than UFPMMax in Top-K mode across all three datasets and all values of K : roughly **3–7 $\times$ faster on mushroom**, **5–30 $\times$ faster on retail**, and **7–11 $\times$ faster on accidents**. The advantage is largest on retail (e.g. 0.6 s vs. 16.6 s at $K=5$) because UFPMMax must build expensive projections over 16K items while UGenMax’s dynamic threshold raising prunes most of them upfront.

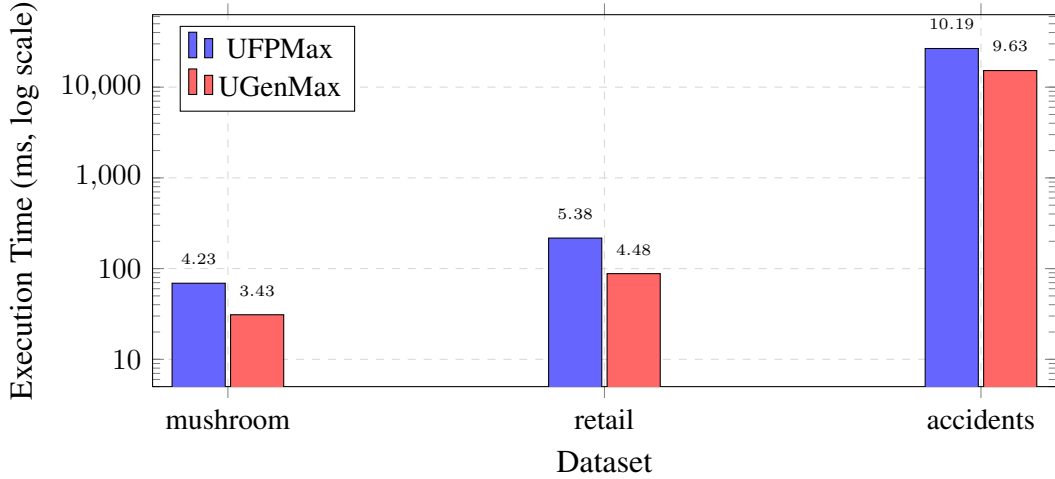


Figure 6.4: Cross-dataset scalability: UFPMax vs. UGenMax (log scale)

Table 6.5: Top-K mode on mushroom_medium (8,416 transactions, 120 items)

K	Time (ms)		Memory (MB)		Final minsup	
	UFPMax	UGenMax	UFPMax	UGenMax	UFPMax	UGenMax
5	908	162	33.9	21.7	1,711.2	2,451.6
10	954	146	48.3	29.5	1,225.8	1,975.7
20	963	176	51.0	32.2	973.7	1,332.9
50	1,178	298	20.8	18.7	613.9	955.1
100	1,323	443	40.5	37.3	477.5	720.8

- Dynamic threshold raising (Pruning 3) is what drives this gap: UGenMax’s final minsup is consistently *higher* than UFPMax’s at the same K (e.g. on mushroom at $K=10$: 1,976 vs. 1,226), which means UGenMax aggressively shrinks the search space during traversal.
- As K increases, both algorithms take longer because the threshold stabilises at a lower value, expanding the search space (e.g. on retail UGenMax goes from 0.6 s at $K=5$ to 4.8 s at $K=100$).
- Memory behaviour mirrors the static-minsup results: UGenMax uses less memory than UFPMax on the sparse retail and the dense mushroom in most cases, but uses noticeably more on very dense accidents, again reflecting the cost of weighted tidsets when most transactions contain most items.

6.7 Summary of Findings

Taking the Chapter 6 results together, three findings stand out. First, correctness holds throughout: UGenMax and UFPMax produce identical MFI sets across all datasets and thresholds. Second, on the primary retail_medium dataset UGenMax is 16–33% faster than UFPMax and

Table 6.6: Top-K mode on retail_medium (88,162 transactions, 16,470 items)

K	Time (ms)		Memory (MB)		Final minsup	
	UFPMax	UGenMax	UFPMax	UGenMax	UFPMax	UGenMax
5	16,604	594	203.9	105.0	936.8	2,248.1
10	17,116	560	123.4	102.0	1,124.0	1,630.4
20	17,315	675	66.3	108.0	646.8	741.7
50	18,135	1,531	106.8	131.0	320.2	519.8
100	23,936	4,755	149.2	101.1	123.7	260.6

Table 6.7: Top-K mode on accidents_medium (340,183 transactions, 468 items). Times in seconds.

K	Time (s)		Memory (MB)		Final minsup	
	UFPMax	UGenMax	UFPMax	UGenMax	UFPMax	UGenMax
5	91.7	10.4	870.4	1,351.9	79,627	137,418
10	93.5	12.1	837.3	829.8	68,709	108,265
20	106.0	14.8	862.7	1,382.9	54,132	75,922
50	133.6	12.4	818.7	1,346.7	35,942	60,157
100	130.3	18.0	1,130.5	1,187.2	30,162	47,455

uses roughly 30–38% less memory — a time advantage that generalises to dense mushroom (37–47% faster on non-trivial workloads) and to very dense accidents (14–49% faster across the full minsup range), although on accidents the weighted-tidset representation becomes larger than UFPMax’s projected databases, so memory savings do not generalise to the very-dense regime. Finally, both algorithms scale to 340K transactions (accidents) within acceptable time limits, and UGenMax additionally outperforms UFPMax in Top-K mode by a substantial margin ($3\text{--}7\times$ on mushroom, $5\text{--}30\times$ on retail, $7\text{--}11\times$ on accidents) thanks to dynamic threshold raising.

CHAPTER 7

CONCLUSIONS

7.1 Summary

This project designed, implemented, and evaluated two algorithms for mining frequent maximal itemsets from uncertain transaction databases under the expected support model. UGenMax (bottom-up weighted tidset intersections, adapted from GenMax) and UFPMax (top-down conditional database projection, adapted from FPMax) both use branch-and-bound with three pruning strategies: anti-monotonicity, progressive focusing, and dynamic Top-K threshold raising. Both algorithms also support a Top-K mode.

Experiments on three benchmark datasets (mushroom, retail, accidents) at medium uncertainty, with retail_medium as the primary benchmark, demonstrate that both algorithms produce correct results and exhibit distinct performance characteristics.

7.2 Key Contributions

The project’s main contributions are as follows. We formalised the problem with precise mathematical definitions for the expected support model. We adapted GenMax and FPMax for uncertain data, equipping both with a Top-K mode and dynamic threshold raising. The whole code base is pure Java SE with no external dependencies, organised into algorithm classes (no main) and experiment classes (with main and statistics output). Finally, we carried out a comprehensive experimental comparison across three datasets, two algorithms, and varying parameter configurations at medium uncertainty, showing that UGenMax consistently outperforms UFPMax in both static-minsup and Top-K modes.

7.3 Limitations

Several limitations remain. Maximality checking uses a linear scan ($O(|MFI|)$ per check), which a trie- or hash-based index could improve. Memory usage on the accidents dataset also reaches roughly 1.5 GB on the densest configurations, which may be restrictive in memory-constrained environments; this is most pronounced for UGenMax, whose weighted-tidset representation grows large when most transactions contain most items.

7.4 Future Work

A natural next step is a diffset optimisation for UGenMax to reduce memory usage, followed by parallel search across independent subtrees for multi-core speedup. The framework could

also be extended to mining closed or sequential patterns from uncertain data, and evaluated on real-world uncertain datasets from sensor networks or noisy data integration.

Bibliography

- [1] Agrawal, R., Imielinski, T., and Swami, A. (1993). Mining association rules between sets of items in large databases. *Proc. ACM SIGMOD*, pp. 207–216.
- [2] Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules. *Proc. VLDB*, pp. 487–499.
- [3] Aggarwal, C.C., Li, Y., Wang, J., and Wang, J. (2009). Frequent pattern mining with uncertain data. *Proc. ACM SIGKDD*, pp. 29–38.
- [4] Bayardo, R.J. (1998). Efficiently mining long patterns from databases. *Proc. ACM SIGMOD*, pp. 85–93.
- [5] Chui, C.K., Kao, B., and Hung, E. (2007). Mining frequent itemsets from uncertain data. *Proc. PAKDD*, pp. 47–58.
- [6] Fournier-Viger, P., et al. (2014). SPMF: A Java open-source pattern mining library. *JMLR*, 15, pp. 3569–3573.
- [7] Gouda, K. and Zaki, M.J. (2005). GenMax: An efficient algorithm for mining maximal frequent itemsets. *Data Mining and Knowledge Discovery*, 11(3), pp. 223–242.
- [8] Grahne, G. and Zhu, J. (2003). Efficiently using prefix-trees in mining frequent itemsets. *Proc. IEEE ICDM Workshop on FIMI*.
- [9] Han, J., Pei, J., and Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proc. ACM SIGMOD*, pp. 1–12.
- [10] Leung, C.K.-S., Mateo, M.A.F., and Brajczuk, D.A. (2008). A tree-based approach for frequent pattern mining from uncertain data. *Proc. PAKDD*, pp. 653–661.
- [11] Leung, C.K.-S. and Tanbeer, S.K. (2013). Fast tree-based mining of frequent itemsets from uncertain data. *Proc. DASFAA*, pp. 272–287.
- [12] Li, H. and Zhang, N. (2019). Mining top-k frequent itemsets from uncertain databases. *Proc. DSAA*.
- [13] Pan, F., Cong, G., Tung, A.K.H., Yang, J., and Zaki, M.J. (2003). CARPENTER: Finding closed patterns in long biological datasets. *Proc. ACM SIGKDD*, pp. 637–642.
- [14] Sun, L., Cheng, R., Cheung, D.W., and Cheng, J. (2010). Mining uncertain data with probabilistic guarantees. *Proc. ACM SIGKDD*, pp. 273–282.

- [15] Chen, S., Nie, L., Tao, X., Li, Z., and Zhao, L. (2020). Approximation of probabilistic maximal frequent itemset mining over uncertain sensed data. *IEEE Access*, 8, pp. 97529–97539.
- [16] Szathmary, L., Napoli, A., and Valtchev, P. (2006). Towards rare itemset mining. *Proc. IEEE ICTAI*.